

Open in app ↗

Sign up

Sign in

Medium

 Search

IMAGE TECHNOLOGY

Noise filtering in Digital Image Processing

Noise is always presents in digital images during image acquisition, coding, transmission, and processing steps.

5 min read · Sep 2, 2018



Anisha Swain

Follow



Listen



Share



Hello People. In our last article named “[Noise in Digital Image Processing](#)” we had promised to get back to you with another article on filtering techniques and filters. So here we are with yet another article of our series “Image Vision” on Noise Filtering.

Noise is always presents in digital images during image acquisition, coding, transmission, and processing steps. It is very difficult to remove noise from the digital images without the prior knowledge of filtering techniques. In this article, a brief overview of various noise filtering techniques. These filters can be selected by analysis of the noise behaviour. In this way, a complete and quantitative analysis of noise and their best suited filters will be presented over here.

Get Anisha Swain's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Filtering image data is a standard process used in almost every image processing system. Filters are used for this purpose. They remove noise from images by preserving the details of the same. The choice of filter depends on the filter behaviour and type of data.

Filtering Techniques:

We all know that, noise is abrupt change in pixel values in an image. So when it comes to filtering of images, the first intuition that comes is to replace the value of each pixel with average of pixel around it. This process smooths the image. For this we consider two assumptions.

Assumption:

1. The true value of pixels are similar to true value of pixels nearby
2. The noise is added to each pixel independently.

Let's first consider 1-dimensional function before going into 2-dimensional image.

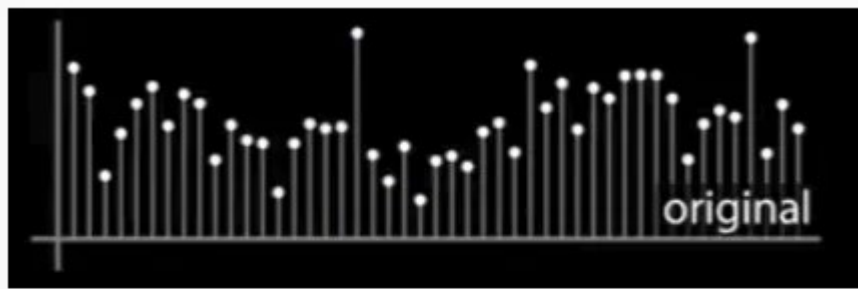


Fig.1 Original image signal in 1 Dimension with noise
Source: Udacity

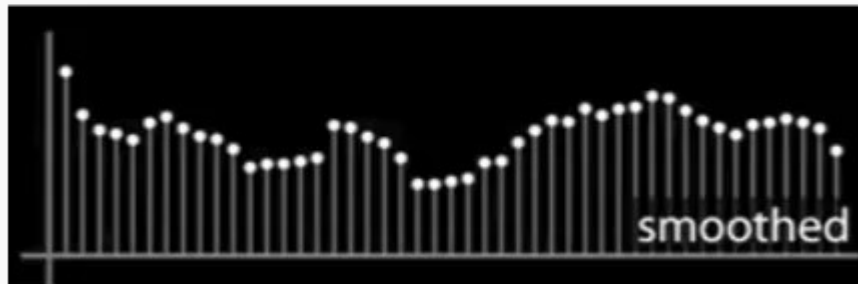


Fig.2 Image signal in 1 Dimension after averaging
Source: Udacity

In the above image of original function(fig-1), if we will consider each circle as pixel values, then the smoothed function(fig-2) is the result of averaging the side by pixel values of each pixel.

1. Filtering with weighted moving average uniform weight:

Instead of just thinking about averaging the local pixel, which is resulting in some loss of data, we consider a set of local pixel and assign them as uniform weights. Here we assume that noise is added to each pixel independently. According to this noise amount, we assign weights to different pixels.

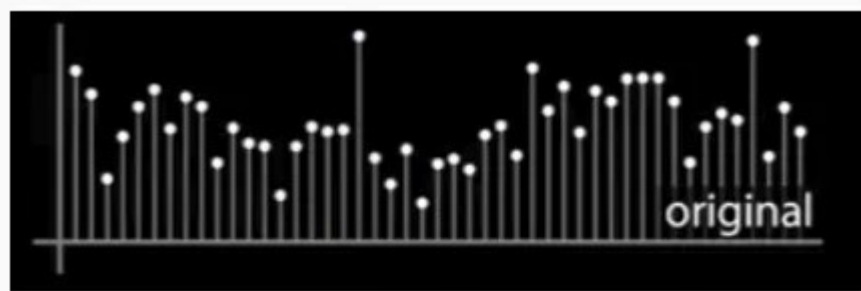


Fig.3 Original image signal in 1 Dimension with noise
Source: Udacity

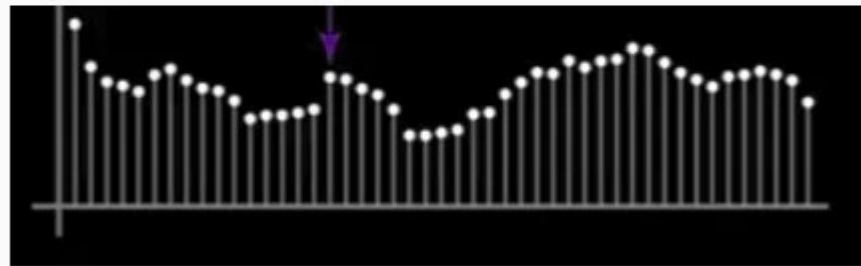


Fig.4 Image signal in 1 Dimension after uniform weighted moving averaging
Source: Udacity

2. Filtering with weighted moving average non-uniform weight

Previously we took the assumption that the true value of pixels are similar to true value of pixels nearby. But it is not always true. So for higher accuracy we assign the nearby pixels with greater weight then the pixels that are far away. This smooths the image and preserves the image information with less amount of data loss.

3. Weighted moving average in 2-dimensional image

Thinking of image as a 2-dimensional matrix, we slide a small window(the red square in fig. 5) over the whole image to replace each pixel with the average of nearby pixels. This small window is otherwise known as **mask or kernel**.

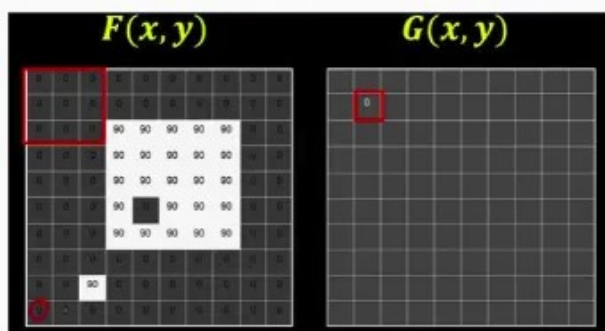


Fig.5 First step for calculating weighted moving averaging in a 2D image matrix
Source: Udacity

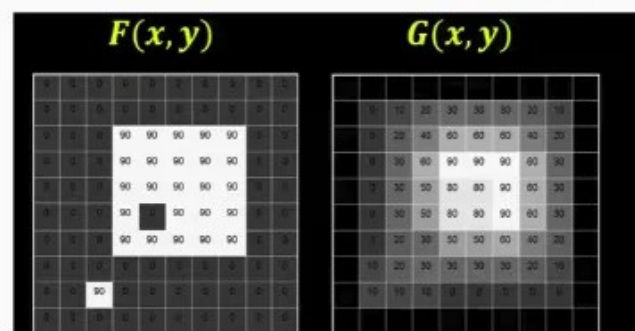


Fig.6 Last step for calculating weighted moving averaging in a 2D image matrix
Source: Udacity

The process used in filtering with uniform weights is also called correlation or **correlation filtering**.

Say the averaging window size is $2k+1 \times 2k+1$:

$$G[i, j] = \frac{1}{(2k+1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]$$

Fig.7 Correlation function for uniform weights. src: Udacity

In correlation filtering with non-uniform weight, an function is used as non-uniform weights which is also called **mask or kernel** (function of the pixel values of the small sliding window) . The process used in it is called **cross-correlation**.

Now generalize to allow **different weights** depending on neighboring pixel's relative position:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{H[u, v]}_{\text{Non-uniform weights}} F[i+u, j+v]$$

Fig.8 Correlation function for non-uniform weights .src: Udacity

Types of Image noise filters:

There are different types of image noise filters. They can typically be divided into 2 types.

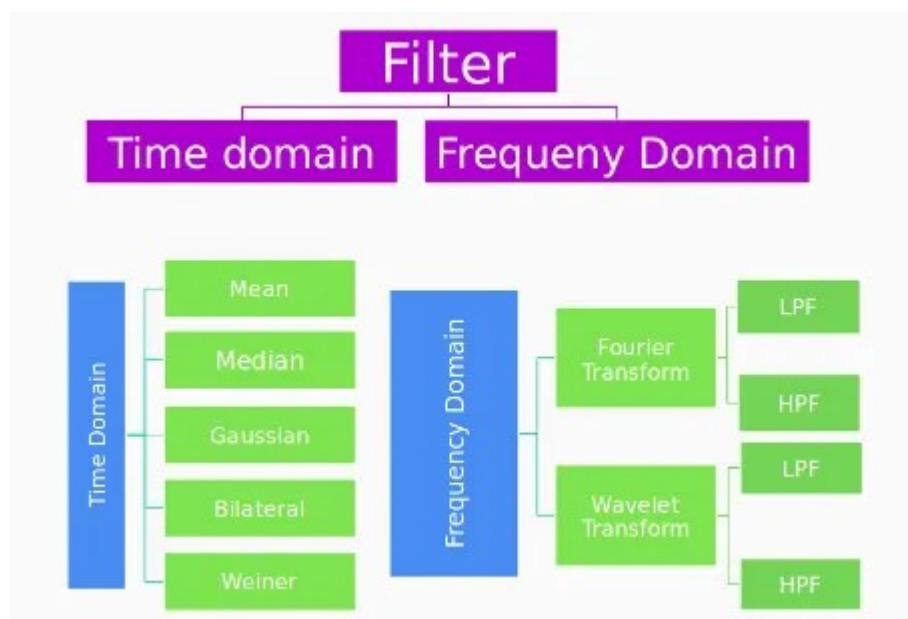


Fig.9 Classification of Filters

Though there are many types of filters, for this article we will consider 4 filters which are mostly used in image processing.

1. Gaussian Filter:

In **image processing**, a **Gaussian blur** (also known as **Gaussian smoothing**) is the result of blurring an **image** by a **Gaussian function** (named after mathematician and scientist Carl Friedrich Gauss). It is a widely used effect in graphics software, typically to reduce **image noise** and reduce detail.

1.1 Implementation of Gaussian Filter with OpenCV and Python:

```
1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('gaussnoise.jpg')
6
7 blur = cv2.GaussianBlur(img,(5,5),0)
8
9 plt.subplot(121),plt.imshow(img),plt.title('Original')
10 plt.xticks([], plt.yticks([]))
11 plt.subplot(122),plt.imshow(blur),plt.title('Averaging')
12 plt.xticks([], plt.yticks([]))
13 cv2.imwrite('gaussfiltered.jpg',blur)
14 plt.show()
15
```



(Filtering Gaussian Noise)

2. Mean Filter:

Mean filter is a simple sliding window that replace the center value with the average of all pixel values in the window. The window or kernel is usually a square but it can be of any shape.



Fig.10 Calculating Mean by sliding window

2.1 Implementation of Mean Filter with OpenCV and Python:


```
1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('poisson.jpg')
6
7 # kernel = np.ones((5,5),np.float32)/25
8 # dst = cv2.filter2D(img,-1,kernel)
9
10 #or
11 blur = cv2.blur(img,(5,5))
12
13 plt.subplot(121),plt.imshow(img),plt.title('Original')
14 plt.xticks([], plt.yticks([]))
15 plt.subplot(122),plt.imshow(blur),plt.title('Averaging')
16 plt.xticks([], plt.yticks([]))
17 cv2.imwrite('poissonfiltered.jpg',blur)
18 plt.show()
19
```



(Filtering Poisson Noise)

3. Median Filter:

Mean filter is a simple sliding window that replace the center value with the Median of all pixel values in the window. The window or kernel is usually a square but it can be of any shape.



Fig.11 Calculating Median by sliding window

3.1 Implementation of Median Filter with OpenCV and Python:

```

1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('poisson.jpg')
6
7 # kernel = np.ones((5,5),np.float32)/25
8 # dst = cv2.filter2D(img,-1,kernel)
9
10 #or
11 blur = cv2.blur(img,(5,5))
12
13 plt.subplot(121),plt.imshow(img),plt.title('Original')
14 plt.xticks([], plt.yticks([]))
15 plt.subplot(122),plt.imshow(blur),plt.title('Averaging')
16 plt.xticks([], plt.yticks([]))
17 cv2.imwrite('poissonfiltered.jpg',blur)
18 plt.show()
19

```



(Filtering Salt and Pepper noise)

4. Bilateral Filter

Bilateral filter uses Gaussian Filter but it has one more multiplicative component which is a function of pixel intensity difference. It ensures that only pixel intensity similar to that of the central pixel is included in computing the blurred intensity value. This filter preserves edges.

```

1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 img = cv2.imread('gaussnoise.jpg')
6
7 blur = cv2.GaussianBlur(img,(5,5),0)
8
9 bilateral = cv2.bilateralFilter(img,9,75,75)
10
11 plt.subplot(121),plt.imshow(img),plt.title('Original')
12 plt.xticks([], plt.yticks([]))
13 plt.subplot(122),plt.imshow(blur),plt.title('Averaging')
14 plt.xticks([], plt.yticks([]))
15 cv2.imwrite('gaussfiltered.jpg',blur)
16
17 plt.show()
18 plt.subplot(122),plt.imshow(bilateral),plt.title('Averaging')
19 plt.xticks([], plt.yticks([]))
20 cv2.imwrite('bilateralfiltered.jpg',bilateral)
21 plt.show()

```




Fig.12 Result of filtering gaussian noise with Bilateral filter



Fig.13 Result of filtering gaussian noise with Gaussian filter

Analysis of best suited filters for noises:

From the implementation of both noise and filters, we analysed best suited filters for different image noises.

NOISE	BEST SUITED FILTERS
Salt and Pepper	Median
Poisson	Mean
Gaussian	Gaussian/BilAtERAL
Speckle	Weiner

With this we conclude this article on noise filtering in image processing. To know more about noise please refer [this](#). For more updates on image processing stay tuned with us and let us know your queries with your comments. Happy Coding!

Let us your thoughts about the article in the comment section below and if you want to connect with me, here I am -> [Twitter](#) or [Linkedin](#) or [Instagram](#)

[Opencv](#)[Image Processing](#)[Computer Vision](#)[Image](#)[Filters](#)



Follow

Written by Anisha Swain

353 followers · 78 following

MTS @Salesforce, Former SE@Red Hat,GHCI18 Scholar,Open Source Contributor, Computer Vision and Deep learning enthusiast. contact:<https://twitter.com/anishaswain>

Responses (6)



Write a response

What are your thoughts?

See all responses